

Data Engineer

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 23: AI Roles and Responsibilities

1. The Data Engineer's Place in AI Security

Data Engineers are the architects of the infrastructure that every other AI role depends on. Data Scientists need clean, labelled training data. ML Engineers need reliable data feeds for training pipelines. AI Governance Engineers need data systems that can enforce handling policies technically. None of these roles can operate securely if the data pipelines beneath them are compromised.

This makes the Data Engineer's security decisions foundational. A poorly secured ingestion pipeline does not just produce bad data — it creates an attack surface that sits upstream of every AI system the organisation operates. Compromise at the pipeline layer propagates silently to every model trained on that data, often without any alert being triggered.

The SecAI+ exam treats Data Engineers as the practitioners responsible for training data security and data pipeline integrity. Understanding what Data Engineers build — and where those systems can fail — is essential for answering scenario questions that describe model behaviour anomalies or data breach events in AI contexts.

2. Core Data Engineering Tooling and What Attackers Target

Tool / Platform	Primary Function	Security Configuration Required	Attacker Interest
Apache Kafka	Streaming event pipeline	TLS between brokers/clients; SASL authentication; ACLs per topic; audit logging	Intercept real-time feeds; inject malicious events into model input streams
Apache Spark	Distributed data processing	Cluster access control; executor network isolation; secure temp storage	Exploit temp storage containing sensitive training data
Apache Airflow / Prefect / Dagster	Pipeline orchestration	Role-based access to DAGs; encrypted connections to external systems; audit logs	Modify DAG logic to redirect data or suppress quality checks
Apache Iceberg / Delta Lake / Apache Hudi	Lakehouse table formats	Storage-level encryption; access control on table metadata; ACID transaction logs	Tamper with historical snapshots to corrupt versioned training datasets
Apache Atlas / AWS Glue Catalog / Amundsen	Data catalog and lineage	Access control on metadata (not just data); audit access to sensitive classifications	Discover sensitive data locations without accessing data directly
Snowflake / BigQuery / Redshift	Data warehouse / analytics store	Network policies; least-privilege RBAC; query audit logging; column-level security	Exfiltrate training datasets; query sensitive columns without authorisation

3. Data Quality as a Security Property

Data quality is most often framed as an operational concern — poor quality degrades model performance. It is also a security property. If an attacker can degrade data quality in a training pipeline without triggering quality alerts, they can influence model behaviour without ever touching the model itself.

This attack pattern is documented in MITRE ATLAS v2.1 under AML.T0020 (Poison Training Data). It exploits weak data quality controls in ingestion pipelines. The attacker may inject statistical outliers, corrupt label assignments, or introduce systematic bias into a specific demographic slice of the training data — causing the resulting model to make predictably wrong decisions on a targeted input class.

Security principle: Data quality controls — schema validation, referential integrity checks, anomaly detection on incoming batches, and statistical process control — serve both operational and security purposes simultaneously. Data Engineers who implement these controls are building a first line of defence against training data poisoning.

Anomaly detection on incoming data is particularly important. A single malicious batch that deviates statistically from the historical distribution should trigger review. Without this gate, poisoned data passes through as silently as legitimate data.

4. Named Incident: Data Pipeline Injection Attack (2024)

Data Pipeline Injection — ML Training Infrastructure (2024): A research team demonstrated a pipeline injection attack in which a scheduled ETL job pulled data from an external API without validating the API response format. The attacker first compromised the external API provider via credential stuffing, then injected malicious records into the victim organisation's training dataset. The injected records were crafted to introduce a backdoor into the fraud detection model trained on this data — causing it to systematically approve transactions matching a specific pattern the attacker could reproduce. The attack bypassed all model-level security controls because it operated below the model layer. The required defences were input validation at the ingestion boundary, cryptographic verification of external data sources, and statistical anomaly detection on incoming batches.

This incident illustrates the broader principle: pipelines that trust external data sources unconditionally create an attack surface that bypasses all downstream controls. The model, the deployment infrastructure, and the inference API can all be secured correctly while remaining vulnerable to upstream pipeline compromise.

5. Data Lineage: The Audit Trail AI Incident Response Depends On

Data lineage is the documented record of where each data record came from, what transformations were applied to it, and where it was used as an output. In AI contexts, lineage connects a trained model's behaviour back through the training pipeline to the original data sources.

When a deployed model begins behaving unexpectedly — producing biased outputs, systematically failing on certain input types, or generating results inconsistent with its validation performance — investigators need to trace the model back to its training data. Without lineage, this is guesswork. With lineage, it is an audit.

Lineage Tool	Type	Key Capability for AI Security
Apache Atlas	Open source, metadata management	Tracks data flows across Hadoop ecosystem; integrates with Kafka and Spark
OpenLineage	Open standard (not a tool)	Vendor-neutral lineage metadata format; supported by Airflow, Spark, dbt, Flink
Marquez	Open source, lineage server	Implements OpenLineage spec; API-driven; queryable lineage graph
Alation	Commercial catalog + lineage	Automated lineage discovery; policy enforcement integration
Collibra	Commercial governance platform	Enterprise-grade lineage with policy, classification, and stewardship workflows

ISO/IEC 42001:2023 — the AI management system standard — requires documentation of training data provenance. Data lineage systems are the technical implementation of that requirement. Data Engineers who treat lineage as optional overhead are eliminating the forensic capability that AI incident response depends on.

6. Streaming vs. Batch: Security Profiles Differ

The architectural choice between streaming and batch data processing is not only a performance decision — it creates distinct security profiles that Data Engineers must manage explicitly.

- Streaming pipelines (Kafka, Flink, Kinesis): Process events at millisecond-to-second latency. Data is ephemeral — if security logging is not built into the pipeline from the start, individual events may be processed and discarded without any audit trail. Stream processing is harder to audit retrospectively.
- Batch pipelines (Spark jobs, scheduled ETL): Process data in scheduled windows. Inputs and outputs persist as files, making them easier to audit. However, they create windows of staleness between updates — for fraud detection models, a batch pipeline updated nightly may miss attack patterns that emerged since the last batch run.
- Security implication for AI: For AI systems where data freshness directly affects security effectiveness — fraud detection, threat intelligence models, anomaly detection — the streaming vs. batch choice affects the model's ability to detect attacks that occurred recently. Data Engineers and AI Architects make this choice jointly, with Data Engineers accountable for the security controls appropriate to whichever approach is selected.

7. Red Team Perspective: How Attackers Target Data Pipelines

Red teams targeting data infrastructure probe for a consistent set of weaknesses. Understanding the attacker perspective helps Data Engineers prioritise defensive configuration.

- Ingestion trust: Pipelines that pull from external APIs or public data repositories without validating response format, size, or statistical properties. These are the primary vector for data poisoning at scale.
- SQL/code injection in transformation logic: Pipelines that construct queries dynamically from data field values are vulnerable to injection — a malicious value in an incoming record can modify the transformation query itself.
- Unencrypted data in transit: Pipelines that move sensitive data between systems without TLS. This is particularly common in internal network segments where encryption is assumed to be unnecessary.
- Excessive-permission service accounts: ETL jobs running with database admin credentials when read access to specific tables would suffice. Compromising one service account should not grant lateral access to unrelated data stores.
- Data store misconfiguration: Publicly accessible object storage buckets, unencrypted database backups, or database management interfaces exposed to the internet. These are persistent findings in cloud data infrastructure.
- Metadata as an attack surface: Data catalog systems that expose metadata without access control allow an attacker to discover what sensitive data exists and where — without accessing the data itself.

8. Data Governance Implementation: Policy Into Pipeline

Data governance policies state what should happen to data. Data Engineers implement the technical controls that make those policies effective. A data handling policy that is not implemented in pipeline configuration does not function in practice, regardless of how clearly it is stated in documentation.

Key technical governance controls Data Engineers implement:

- Column-level security: Restricts access to specific columns containing sensitive fields (e.g., PII, PHI) without restricting access to the broader table. Implemented natively in Snowflake, BigQuery, and Redshift.
- Row-level security: Limits record visibility based on user attributes — for example, a regional analyst can only query records from their assigned geography.
- Data masking: Replaces sensitive values with masked equivalents in non-production environments. Prevents real PII from appearing in development, test, and analytics environments.
- Automated retention and deletion: Scheduled processes that delete records past their documented retention period and propagate deletions to all downstream stores — required for GDPR Article 17 compliance.
- Data classification tagging: Metadata labels (PII, PHI, CONFIDENTIAL) applied at ingestion that governance systems use to route data to appropriate handling controls automatically.
- Purpose limitation enforcement: Access controls that restrict which downstream use cases can consume data from each pipeline — technically enforcing purpose boundaries rather than relying on practitioner self-regulation.

9. Analogy: The Data Engineer as Water Treatment Plant Operator

Think of the data engineer as the operator of a water treatment plant. Every building downstream depends on water that arrives clean, at sufficient pressure, and without contaminants. The building owners install their own security systems, fire suppression, and internal plumbing — but none of those controls matter if the water supply itself is compromised.

In AI systems, the 'water' is training data and inference inputs. The Data Engineer controls the treatment process — filtration (schema validation), quality monitoring (anomaly detection), and lineage tracking (chain of custody). A breach at the treatment plant level contaminates everything downstream. The model, the deployment pipeline, and the inference API are the building's systems — they depend on the data supply being trustworthy before any of their own controls can be effective.

This analogy reinforces the exam-relevant principle: AI security begins with data security, and data security is the Data Engineer's domain.

10. Regulatory Requirements Data Engineers Must Implement

Regulation	Key Requirement	Data Engineering Implementation
GDPR Article 17	Right to erasure — personal data must be deleted on request	Deletion propagation across all pipeline stages and downstream stores; no orphaned copies in backups or analytics layers
GDPR Article 20	Data portability — personal data must be exportable in portable format	Export APIs or batch processes for individual record retrieval from analytics-optimised stores
CCPA	Consumer rights to access, delete, and opt out of sale of personal information	Similar to GDPR; requires identifiable records across all pipeline stages
HIPAA	PHI audit logging, access controls, and breach notification	Comprehensive access logging on all PHI-containing stores; access control aligned to minimum necessary standard
ISO/IEC 42001:2023	Documentation of training data provenance	Implemented via data lineage systems; lineage must trace each training record to its source

Data Engineers who design pipelines without understanding these regulatory requirements discover compliance gaps during audits. Remediation after deployment — adding deletion propagation, building export capabilities, retrofitting audit logging — is consistently more expensive and disruptive than implementing these controls at design time.

11. Responsible Practice: The Ethics of Pipeline Design

Data Engineers hold significant power over what data gets collected, retained, and made available for AI training. Responsible practice requires active engagement with the ethical dimensions of that power — not a purely technical orientation that defers all policy decisions to others.

- Challenge data requests that appear to exceed legitimate business purpose — collecting more fields or retaining data longer than the use case requires creates unnecessary risk for data subjects.

- Raise concerns about collection of sensitive data categories without appropriate consent mechanisms — particularly when pipelines are designed to ingest data from public sources or third-party providers.
- Refuse to implement pipelines that would enable surveillance or tracking beyond what applicable law and organisational policy permits.
- Flag data sharing arrangements that may violate data subject rights — particularly cross-border transfers that may not have appropriate legal basis.

Data Engineers who adopt a 'I just build the pipelines, policy is someone else's concern' stance are making an active choice not to engage with ethical dimensions that their technical decisions directly shape. The pipelines they build encode those choices at infrastructure level, making them persistent and difficult to reverse.

12. Key Terms Glossary

Term	Definition
ELT / ETL	Extract-Load-Transform / Extract-Transform-Load: workflows that move data from source systems to analytical or training data stores, performing transformations at different stages
Data Lakehouse	Architecture combining the scalability of a data lake with the ACID transaction support of a data warehouse; implemented via table formats such as Apache Iceberg, Delta Lake, and Apache Hudi
Data Lineage	The documented record of a data record's origin, transformation history, and downstream usage; essential for AI incident investigation and ISO/IEC 42001:2023 compliance
Data Catalog	A system that tracks metadata, classification, and lineage for data assets across an organisation; examples include Apache Atlas, AWS Glue Catalog, and Alation
AML.T0020	MITRE ATLAS identifier for the Poison Training Data attack technique — adversarial manipulation of training data to influence model behaviour
ACID transactions	Atomicity, Consistency, Isolation, Durability — database properties that guarantee reliable data modification; critical for maintaining training dataset integrity under concurrent writes
Column-level security	Access control that restricts access to specific columns in a table rather than the full table; used to protect sensitive fields such as PII or PHI
Data masking	Replacing sensitive data values with realistic but fictional equivalents in non-production environments; prevents real PII from appearing in development or analytics contexts
OpenLineage	An open standard for lineage metadata interchange; allows lineage information to flow between different tools such as Airflow, Spark, and dbt without proprietary lock-in
Purpose limitation	The data governance principle that data collected for one purpose should not be used for another without appropriate authorisation; technically enforced via pipeline access controls
Deletion propagation	The process of ensuring that a deletion request is applied to all copies of a data record across every pipeline stage and downstream store —

	required for GDPR Article 17 compliance
Statistical process control	Quality monitoring technique that uses statistical methods to detect when a process is producing outputs outside expected parameters; applied to data pipelines to detect anomalous batches

Quick Revision Checklist

- Data Engineers build ELT/ETL pipelines, data lakehouses, orchestration, catalogs, and streaming infrastructure — the foundation beneath all AI systems.
- Data quality controls (schema validation, anomaly detection, statistical checks) serve both operational and security purposes — they are the first defence against training data poisoning.
- MITRE ATLAS AML.T0020 (Poison Training Data) targets weak quality controls in ingestion pipelines — understand the attack pattern and the pipeline controls that detect or prevent it.
- Data lineage connects model behaviour back to training data sources — without lineage, AI incident response is guesswork; ISO/IEC 42001:2023 requires training data provenance documentation.
- Streaming and batch architectures create different security profiles — streaming data is ephemeral and harder to audit; batch data is persistent but creates staleness windows.
- Red team targets: ingestion trust gaps, injection in transformation logic, unencrypted data in transit, excessive-permission service accounts, misconfigured data stores, unprotected catalog metadata.
- Technical governance controls: column-level security, row-level security, data masking, automated retention/deletion, classification tagging, and purpose limitation enforcement.
- Regulatory requirements: GDPR Articles 17 and 20 (erasure and portability), CCPA consumer rights, HIPAA PHI controls, ISO/IEC 42001:2023 provenance requirements.
- Responsible practice: Data Engineers should challenge excessive data collection, refuse pipelines that enable unlawful surveillance, and flag data sharing with inadequate legal basis.
- Exam framing: SecAI+ tests understanding that AI security begins with data security — scenario questions may describe model anomalies and ask which pipeline control would have prevented them.

10 Exam-Based MCQs — Data Engineer Role and Pipeline Security

Test yourself after reading. These questions mirror the SecAI+ exam style — scenario-driven, with plausible distractors drawn from adjacent concepts. Read every explanation, including for questions you answered correctly.

Q1. *Scenario:* A financial services firm uses a batch ETL pipeline that ingests transaction records from regional databases nightly. The pipeline applies schema validation and type checking but no statistical quality checks. The fraud detection model trained on this data has been in production for three months. During an investigation into unexpectedly biased outputs from a deployed fraud detection model, the security team determines that the model's training dataset contains a systematic under-representation of legitimate transactions from a specific geographic region. No anomaly was flagged during training. Which data pipeline control would MOST likely have detected this issue before the model was trained?

- A) TLS encryption on all data transport between pipeline stages

- B) Statistical process control checks on incoming data batches — flagging distributions that deviate from historical norms
- C) Column-level security restricting access to the geographic field
- D) Data masking applied to transaction records in the non-production environment

Answer: B **Explanation:** B is correct because statistical process control on incoming batches would flag the unusual geographic distribution as a deviation from historical patterns, triggering review before the biased data entered the training dataset. A is wrong because encryption protects data in transit but does not detect quality or distributional anomalies in the data itself. C is wrong because restricting access to a column does not validate its content or detect under-representation. D is wrong because data masking is applied in non-production environments for privacy protection — it does not analyse incoming data distributions.

Q2. Scenario: During a red team engagement, the team successfully exfiltrates a table of 40,000 employee salary records by compromising a single ETL service account credential. The ETL job's documented function is to extract customer transaction records for ML model training. A red team engagement against a retail company's AI infrastructure reveals that the ETL service account used to load data into the ML training warehouse has SELECT permissions on all tables in the production database, including tables containing employee PII not required for model training. Which principle does this finding violate, and what is the correct remediation?

- A) Data minimisation — remediate by implementing data masking on PII columns
- B) Least privilege — remediate by restricting the ETL service account to SELECT on only the tables required for training data extraction
- C) Purpose limitation — remediate by deleting the employee PII from the data warehouse
- D) Data integrity — remediate by adding referential integrity checks to the ETL pipeline

Answer: B **Explanation:** B is correct because the finding is a least privilege violation — the service account holds permissions beyond what its function requires. The remediation is to scope its access to only the tables necessary for training data extraction. A is wrong because data masking addresses data visibility in non-production environments, not service account permission scoping. C is wrong because purpose limitation addresses use of data for unauthorised purposes — the finding is about access scope, not data use. D is wrong because referential integrity checks validate data relationships, not service account permissions.

Q3. Scenario: The organisation's GDPR compliance officer submits a deletion request on behalf of a data subject. The primary database team confirms the record has been deleted within 72 hours. Three weeks later, an internal audit identifies the record in three downstream tables and an ML training lakehouse snapshot. An AI governance audit finds that the organisation's GDPR deletion process removes personal data records from the primary transaction database within the required timeframe, but that copies of the same records persist in three downstream data warehouse tables and one Apache Iceberg lakehouse snapshot used for ML training. Which capability gap does this represent, and which role is primarily accountable for addressing it?

- A) Missing consent management — AI Governance Engineer

- B) Missing deletion propagation across the data pipeline — Data Engineer
- C) Missing data lineage documentation — AI Auditor
- D) Missing data minimisation controls — AI Risk Analyst

Answer: B **Explanation:** B is correct because the failure is specifically deletion propagation — the absence of a mechanism to propagate deletion requests from the source system to all downstream pipeline stages and stores. This is a Data Engineer accountability. A is wrong because consent management addresses whether data was lawfully collected — the audit finding is about deletion, not collection. C is wrong because lineage documentation tracks where data flows; while lineage would help identify downstream copies, the capability gap is the missing deletion mechanism itself. D is wrong because data minimisation is about collecting only necessary data — the finding is about incomplete deletion after the retention period.

Q4. Which MITRE ATLAS technique describes an attack in which malicious records are injected into a training dataset to cause a model to behave incorrectly on a specific targeted input class?

- A) AML.T0043 — Craft Adversarial Data
- B) AML.T0020 — Poison Training Data
- C) AML.T0031 — Erode ML Model Integrity
- D) AML.T0006 — Create Proxy ML Model

Answer: B **Explanation:** B is correct. AML.T0020 (Poison Training Data) specifically describes adversarial manipulation of training datasets to cause a model to produce incorrect outputs on targeted inputs. A is wrong — AML.T0043 (Craft Adversarial Data) relates to adversarial examples crafted to fool a model at inference time, not attacks on the training dataset. C is wrong — AML.T0031 does not appear in MITRE ATLAS v2.1. D is wrong — AML.T0006 (Create Proxy ML Model) describes an attacker training a substitute model using query access to the target, not training data poisoning.

Q5. A Data Engineer is designing an ingestion pipeline that will pull customer review data from a third-party sentiment data provider via API. The pipeline will feed a product recommendation model. Which security control should be prioritised at the ingestion boundary?

- A) Encrypting the recommendation model weights at rest
- B) Validating API response format, size, and statistical properties before accepting data into the pipeline
- C) Implementing row-level security on the recommendation model's output table
- D) Adding query audit logging to the data warehouse that stores model outputs

Answer: B **Explanation:** B is correct because the primary risk at the ingestion boundary from a third-party API is that the data may be malformed, statistically unusual, or deliberately manipulated. Input validation at ingestion — format, size, and statistical checks — is the control that addresses this risk. A is wrong because encrypting model weights protects the model itself at rest, not the training data supply. C is wrong because row-level security on output tables controls access to model outputs, not the integrity of input data. D is wrong because audit logging on the output warehouse monitors access after the fact — it does not prevent poisoned data from entering the pipeline.

Q6. What is the primary security advantage of data lineage for AI systems, as it relates to the ISO/IEC 42001:2023 standard?

- A) Lineage encrypts training data at rest, satisfying ISO/IEC 42001:2023 confidentiality requirements
- B) Lineage documents training data provenance, enabling investigators to trace model behaviour anomalies back to specific data sources
- C) Lineage restricts access to training data, satisfying ISO/IEC 42001:2023 access control requirements
- D) Lineage validates that training data was collected with appropriate consent, satisfying ISO/IEC 42001:2023 ethics requirements

Answer: B **Explanation:** B is correct. Data lineage provides a documented chain of custody from model behaviour back to data sources. ISO/IEC 42001:2023 requires documentation of training data provenance, and lineage systems are the technical implementation of that requirement. When a model anomaly occurs, lineage enables forensic investigation. A is wrong — lineage is a metadata tracking capability, not an encryption mechanism. C is wrong — access control is a separate control; lineage tracks data flows, not who can access what. D is wrong — consent management is a separate domain; lineage does not validate how data was collected.

Q7. A streaming data pipeline processing real-time transactions for a fraud detection AI system is found to have no audit logging on individual events. Events are processed and discarded in under 50 milliseconds. What is the security implication?

- A) The pipeline is vulnerable to SQL injection because streaming pipelines do not support parameterised queries
- B) Individual fraudulent transactions that were incorrectly approved by the model cannot be reconstructed for incident investigation
- C) The pipeline cannot achieve ACID transaction guarantees, making it unsuitable for AI training data
- D) The pipeline violates the data minimisation principle because it processes more records than necessary

Answer: B **Explanation:** B is correct. The core implication of missing audit logging in streaming pipelines where events are ephemeral is that individual events cannot be reconstructed after the fact. If the fraud model approves a malicious transaction, there is no record of the specific event that was processed, preventing forensic investigation. A is wrong — SQL injection vulnerability relates to how queries are constructed, not to whether audit logging exists. C is wrong — ACID guarantees relate to data consistency during writes; streaming pipelines can operate without ACID but this is not the implication of the missing logging. D is wrong — data minimisation is about collection scope, not about logging.

Q8. Which data engineering control BEST enforces the data governance principle of purpose limitation at the technical level?

- A) Data masking in non-production environments
- B) Statistical anomaly detection on incoming batches
- C) Pipeline access controls that restrict which downstream use cases can consume data from each pipeline
- D) Automated deletion of records past their retention period

Answer: C **Explanation:** C is correct. Purpose limitation means data collected for one purpose should not be used for another without appropriate authorisation. Pipeline access controls that technically restrict which downstream systems or use cases can consume data from a given pipeline enforce this principle at infrastructure level — preventing practitioners from consuming data for unauthorised purposes even if they wanted to. A is wrong — data masking is a privacy protection for non-production environments; it does not restrict what purposes data can be used for. B is wrong — anomaly detection is a data quality and security control, not a purpose limitation mechanism. D is wrong — automated deletion enforces data retention policy, not purpose limitation.

Q9. An AI Architect is designing a fraud detection system and must choose between a streaming pipeline (sub-second latency) and a nightly batch pipeline. The primary security consideration is the model's ability to detect fraud attack patterns that emerge within hours of first appearing. Which approach is indicated and why?

- A) Batch pipeline — easier to audit because inputs and outputs persist as files
- B) Streaming pipeline — sub-second latency ensures the model's inference inputs reflect attack patterns as they emerge
- C) Either approach is equivalent from a security perspective — the choice is purely operational
- D) Batch pipeline — ACID transaction guarantees make it more resistant to data poisoning

Answer: B **Explanation:** B is correct. When the security effectiveness of an AI system depends on data freshness — specifically the ability to detect patterns that emerge within hours — streaming architecture is indicated. A nightly batch pipeline would miss attack patterns that emerged and were exploited in the hours since the last batch. A is wrong — while auditability is a valid consideration for streaming vs. batch choice, it is not the primary security factor when data freshness is the stated requirement. C is wrong — streaming and batch create materially different security profiles; the choice is not equivalent from a security perspective in time-sensitive contexts. D is wrong — ACID guarantees relate to data consistency during writes, not to resistance against data poisoning or to data freshness.

Q10. Scenario: A Data Engineer at a professional services firm receives a Jira ticket from the HR Analytics team requesting a pipeline that aggregates GPS coordinates from company-issued mobile devices at 15-minute intervals. The ticket states the purpose as 'understanding workplace productivity patterns'. A Data Engineer receives a request to build a pipeline that will aggregate location data from employee mobile devices and make it available to the HR analytics team for productivity research. The Data Engineer is concerned that the data could be used to monitor individual employee movements beyond what the stated research purpose requires. What is the MOST appropriate action?

- A) Build the pipeline as requested — policy decisions are outside the Data Engineer's scope
- B) Build the pipeline with column-level security that restricts access to raw location coordinates

- C) Raise concerns about the request with the AI Governance Engineer and Data Privacy Officer, and decline to implement until the purpose, consent, and access scope are clarified
- D) Build the pipeline but implement data masking on the location fields to prevent individual identification

Answer: C **Explanation:** C is correct. This scenario raises legitimate concerns about purpose limitation (using location data for purposes beyond stated research), consent (whether employees have consented to this level of monitoring), and proportionality (whether individual-level location data is necessary for productivity research). The appropriate responsible practice is to raise these concerns with the relevant governance stakeholders — the AI Governance Engineer and Data Privacy Officer — and not implement until those questions are answered. A is wrong — responsible Data Engineering practice explicitly requires engaging with ethical dimensions of data decisions, not deferring them entirely. B is wrong — implementing with column-level security assumes the pipeline should be built; the first question is whether it should be built at all. D is wrong — data masking addresses privacy in non-production environments; building the pipeline with masked fields still creates the surveillance infrastructure, just with an access layer that could be modified.